

Movie recommendation engine — project vision

Status: planning / design phase. No code has been written yet. This document is the agreed design — give it to any AI assistant before asking for help, so context doesn't need to be re-explained.

What we're building

A personal, content-based movie recommendation system for a single user (a horror-genre enthusiast, though the design generalizes to any genre). It stores a personal viewing archive, predicts how much the user will enjoy any movie (in the archive or brand new), and suggests random movies by genre. Everything runs locally and is fully exportable as one SQLite file — no scraping, no paid APIs, no hosting costs.

Core requirements

- 1. Archive:** Log every watched movie with a personal rating, watch date, and free-form notes (mood, who watched with, rewatch flag). Movie metadata (genre, year, country, runtime, keywords, TMDB rating) is fetched once from TMDB and cached locally.
- 2. Predict:** Given any movie title (in the archive or not), look up its metadata and return a 0-100% “match score” against the user's taste profile, plus a short explanation of which features drove the score.
- 3. Discover:** Given a genre, pull candidate movies, score them all, and surface a weighted-random pick — optionally favoring movies the crowd under-rated but that fit the user's profile (“hidden gems”).

Design principles

- Content-based filtering, not collaborative — recommendations come only from this user's own ratings.
- Similarity scoring (cosine similarity between feature vectors), not a trained ML model — appropriate for a small personal dataset (hundreds of movies) and fully explainable.
- Metadata only from TMDB's official API — no scraping (a comparable open-source project was shut down for exactly this reason).
- Single-user, local-first — SQLite file is the entire database, trivially backed up or exported.
- A local LLM (via LM Studio) handles small natural-language tasks only; it never makes the scoring decision.

Tech stack

- **Backend:** Node.js + Express
- **Database:** SQLite (better-sqlite3)
- **Frontend:** simple web UI (HTML/CSS/JS or lightweight React)
- **Movie metadata:** TMDB API (free, official)
- **Local AI:** LM Studio running LFM2.5-1.2B, OpenAI-compatible API at localhost:1234

Database schema

Two tables. **movies** is a local cache of TMDB data (one row per movie, fetched once). **watched** is the personal log (one row per viewing, so rewatches are additional rows). Relationship: movies.tmdb_id → watched.tmdb_id (1-to-many).

movies field	Description
tmdb_id (PK)	TMDB movie ID
title	Movie title
release_year	Year released
runtime	Runtime in minutes

country	Country of origin
genres	JSON array of TMDB genre names
keywords	JSON array of TMDB keywords/tags
tmdb_rating	TMDB average rating (0-10)
tmdb_votes	TMDB vote count

watched field	Description
id (PK)	Row ID
tmdb_id (FK)	Links to movies.tmdb_id
user_rating	User's rating (0-10 scale)
rating_diff	user_rating minus tmdb_rating (auto-computed)
watch_date	Date watched
watched_with	e.g. alone, friends, family
rewatch	true/false
notes	Free-form text

Recommendation engine logic

Each movie becomes a feature vector: genre one-hot (TMDB's ~19 genres), keyword one-hot (vocabulary built dynamically from this user's archive), country/region, release era, runtime, and TMDB rating — all normalized. The user's **taste profile** is the average feature vector of movies rated highly (e.g. 8+/10). The **match score** for any movie is the cosine similarity between its vector and the taste profile, scaled to 0-100%. **rating_diff** is used for the taste-insights page and can bias discovery toward under-rated movies that still match the profile.

Local LLM (LM Studio) — three jobs only

- **Natural-language search:** turn a free-text query (“something slow and atmospheric, not gory”) into structured filter values.
- **Gap-filling tags:** when TMDB keywords are too sparse, classify tone/pacing from the movie's synopsis.
- **Explanations:** turn a feature-match breakdown into one readable sentence (“why this scored 82%”).

Build roadmap (status: not started)

1. Project skeleton — Express + SQLite, basic routes
2. Movie lookup + add-to-archive via TMDB, including notes/context fields
3. Archive view — browse, filter, sort the log
4. Feature vectorization + match engine (cosine similarity, taste profile)
5. “Check this movie” prediction feature, with score breakdown
6. Random discovery by genre, including hidden-gem bias
7. Taste insights page — genre/era averages, rating differential
8. LM Studio integration — natural-language search + explanations
9. Import/export
10. UI polish + end-to-end testing

Estimated total: ~12-17 working days (roughly 3-6 weeks part-time).